

Hadoop Distributed Systems Lab Final Study Guide

Learn this in order: HDFS commands and architecture; MapReduce data flow; WordCount; combiners and custom Writable values; YARN; job debugging and fault tolerance. Your goal is to run every provided job and explain every important line.

1. HDFS architecture

HDFS stores a file as large blocks distributed across DataNodes. Blocks allow parallel storage and processing, replication, and recovery. Hadoop 3 commonly uses a 128 MB block size, although it is configurable.

NameNode stores metadata: directory tree, file permissions, file-to-block mapping, and DataNode locations. DataNodes store actual block replicas and report heartbeats and block reports. Clients ask the NameNode for locations but transfer data directly with DataNodes.

Replication factor is the number of copies per block. Production often uses 3, improving availability across disk/node/rack failures. A one-node lab cluster commonly uses factor 1 because there are no alternate nodes.

Secondary NameNode creates checkpoints by merging fsimage and edit logs. It is NOT a backup/failover NameNode. High availability instead uses active/standby NameNodes and shared edit-log coordination.

2. YARN

YARN separates resource management from programming models. The ResourceManager schedules resources cluster-wide. A NodeManager runs on each machine and launches/monitors containers. An ApplicationMaster coordinates one submitted application and requests containers for its tasks.

A container is an allocation of resources such as memory and vcores on a node. Hadoop seeks data locality: run map tasks where their HDFS input blocks are stored to reduce network reads.

3. MapReduce pipeline

InputFormat creates logical splits and RecordReader makes records. TextInputFormat normally provides mapper records as (byte offset, line of text).

Map emits zero or more intermediate key/value pairs. WordCount maps each token to (word, 1). Hadoop buffers, partitions, sorts, and spills map output locally.

Partitioning assigns each key to a reducer, normally using hash(key). Equal keys always go to the same reducer. Shuffle transfers each reducer's partition from all mappers. Sort/group gathers all values for one key.

Reduce is called once per distinct key as `reduce(key, Iterable<values>)`. `WordCount` receives `(word, [1,1,...])` and emits `(word,total)`. Each reducer writes one part-xxxxx output file. A zero-reducer job is map-only.

4. Code template and types

`Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT>` defines the four boundary types. `Reducer<KEYIN, VALUEIN, KEYOUT, VALUEOUT>` input types must equal mapper output types.

Use Hadoop Writables across framework boundaries: `Text`, `IntWritable`, `LongWritable`, `DoubleWritable`, `NullWritable`. Driver essentials are `setJarByClass`, `setMapperClass`, `setReducerClass`, final output types, input/output paths, and `waitForCompletion`.

If mapper output types differ from reducer final output types, call `setMapOutputKeyClass` and `setMapOutputValueClass`. `TopNWords` requires this because it maps `NullWritable/Text` but reduces `Text/IntWritable`.

5. Combiner: essential exam topic

A combiner is an optional local pre-aggregation step that reduces network traffic. Hadoop may call it zero, one, or multiple times; therefore it must always preserve correctness.

Safe examples are associative and commutative operations such as sum, count, min, and max. `WordCount` can reuse its sum reducer as a combiner.

A naive average combiner is wrong. An average of averages gives unequal-sized groups equal weight. Instead send `(sum,count)`, add both fields locally and globally, and calculate `sum/count` only at the end.

6. Custom Writable: average word length

`AverageWordLengthByLetter` maps each cleaned word to `(first letter, SumCount(length,1))`. The reducer totals sum and count, then formats `average = totalSum / totalCount`.

`SumCount` implements `Writable`: a public no-argument constructor; `write(DataOutput)`; `readFields(DataInput)`. Read fields in exactly the same order used by `write`. Custom `Writable` lets compound data survive Hadoop serialization during shuffle.

7. Top-N pattern

`TopNWords` reads `WordCount` output. Each mapper retains local top candidates, reducing data sent over the network. It emits `NullWritable` so all candidates share one key, and one reducer produces a global Top-N.

`setNumReduceTasks(1)` makes the single global reducer explicit. This is correct but

can become a bottleneck. At scale use multi-stage or distributed top-K designs.

Note: `TreeMap<Integer,String>` overwrites ties; a robust version needs a collection/composite key and a tie-breaker.

8. Command drill

Compile: `javac -classpath $(hadoop classpath) -d classes src/WordCount.java`

Package: `cd classes && jar -cvf ../wordcount.jar . && cd ..`

Upload: `hdfs dfs -mkdir -p /practice/input ; hdfs dfs -put data/sample.txt /practice/input`

Run: `hadoop jar wordcount.jar WordCount /practice/input /practice/wc-output`

Inspect: `hdfs dfs -cat /practice/wc-output/part-r-00000`

Essential HDFS: `hdfs dfs -ls / ; -mkdir -p /dir ; -put local /dir ; -get /dir/file . ; -cat /file ; -cp /src /dst ; -mv /src /dst ; -rm /file ; -rm -r /dir ; -du -h / ; -df -h / .`

Health/debug: `hdfs dfsadmin -report ; hdfs fsck /dir -files -blocks`.

9. Debugging and pitfalls

Hadoop refuses to overwrite an output directory. If rerunning, use a fresh output path or remove only the old output: `hdfs dfs -rm -r /practice/wc-output`. Confirm the path first; do not delete input.

For failure: first use `hdfs dfs -ls` to validate paths, then check jar/class names, driver types, permissions, and terminal/YARN logs. Common errors: missing map-output type configuration, wrong class name, unsafe combiner, output already exists, and assuming output is one globally sorted file with multiple reducers.

10. Viva answers to rehearse

Why same key to same reducer? Aggregation needs all values for a key together; partitioning and shuffle guarantee it.

Is a combiner guaranteed? No. Its result must be correct whether it runs never, once, or many times.

NameNode vs DataNode? NameNode manages metadata; DataNodes store replicas and serve block bytes.

What if a node fails? YARN retries affected tasks; HDFS detects missing DataNodes via heartbeats and re-replicates blocks from surviving replicas.

Why a custom Writable? To serialize a multi-field intermediate value such as (sum,count) correctly and efficiently.

Final 90-minute plan

15 min: draw HDFS and YARN. 20 min: write WordCount from memory and explain each type. 20 min: implement sum/count Writable average. 15 min: run all three repository jobs and fix an output-path error. 10 min: command drill. 10 min: answer viva questions aloud.

Exam checklist: input exists; output path is new; Writable types match; map-output classes set if different; global result has appropriate reducer design; combiner is safe; part-r files and logs are inspected.